
Software Engineering

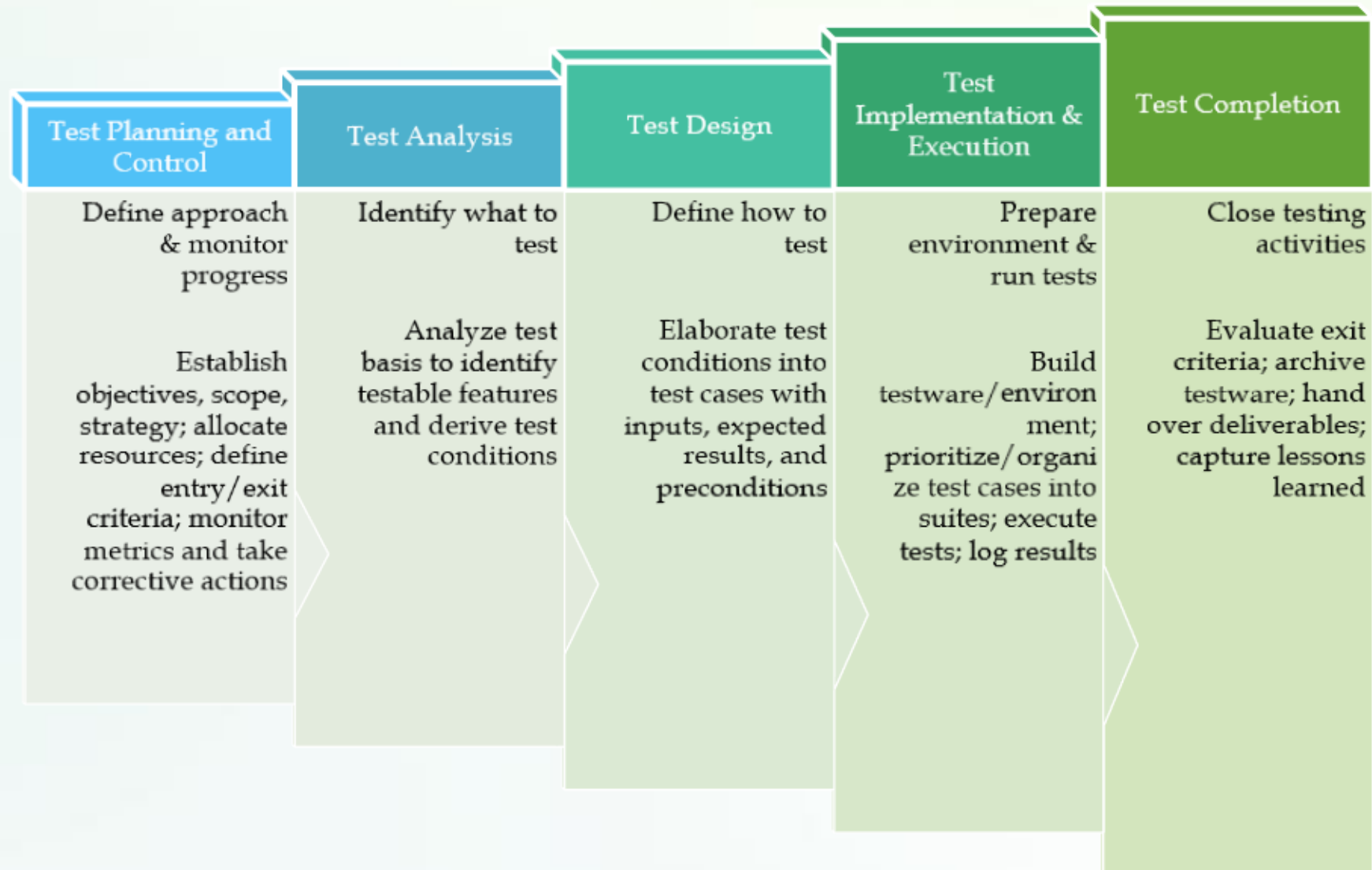
BCSE301L

Module 5 : Validation & Verification

Introduction to Testing

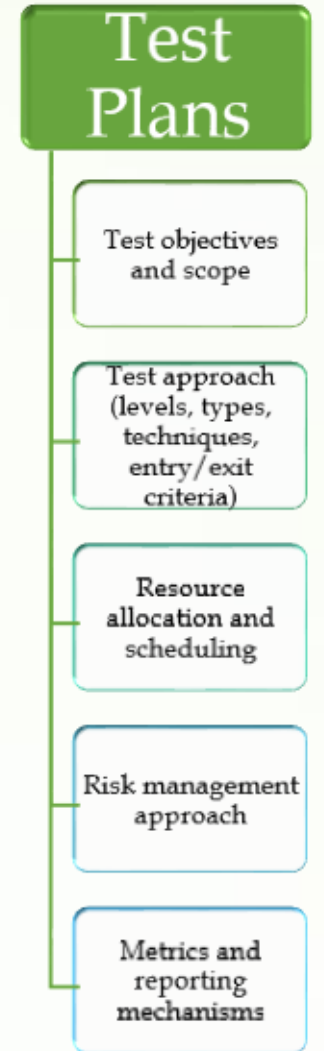
- ❖ The process of **verifying that a system works as intended** and validating that it meets the user's actual needs.
- ❖ It is an **investigative process** conducted to provide stakeholders with information about the quality of the product.
- ❖ Its primary goal is to ensure the delivery of a **high-quality, reliable, and secure product** by **uncovering defects before** the software reaches the end user.

Introduction to Testing - Process



Strategic Approach to Software Testing

- ❖ **Test Strategy:** A description of **how to perform testing** to reach **test objectives** under given circumstances
- ❖ A test strategy provides the **high-level framework** for testing across an organization or program
- ❖ It is documented and implemented through **test plans**, which specify:



Test Plan

- ❖ A test plan is "a document describing the scope, approach, resources, and schedule of intended test activities"
- ❖ It is the primary output of the test planning activity.

Test Plan - Components

Test Objectives

- What quality aspects will be verified; aligned with business goals and risk analysis

Test Scope

- Features *in scope* (to be tested) and *out of scope* (excluded), justified by risk assessment

Test Strategy & Approach

- Test levels to be performed (component, integration, system, acceptance)
- Test types to be applied (functional, non-functional, structural)
- Test techniques (black-box, white-box, experience-based)
- Entry criteria (conditions that must be met before testing begins)
- Exit criteria (measurable conditions to determine when testing is complete)

Test Plan - Components

Resource Planning

- Personnel skills required, test environment specifications, tools needed

Schedule & Milestones

- Timeline aligned with development milestones; dependencies identified

Risk Management Approach

- How product risks (defects in software) and project risks (testing constraints) will be addressed

Metrics & Reporting –

- What will be measured (defect density, test coverage, progress) and reporting frequency

Test Deliverables

- Documentation/artifacts to be produced (test cases, reports, defect logs)

Test Design

- ❖ Test design is the **process of defining the strategy, scenarios, and conditions** required to verify that a system meets its requirements.
- ❖ It acts as a bridge between the high-level test plan and the detailed test cases.

Test Design

Objective

- To achieve maximum test coverage with minimal effort and resources.

Key Activities

- Analyzing requirements, identifying test scenarios (high-level "what to test"), and selecting specific techniques.

Common Techniques

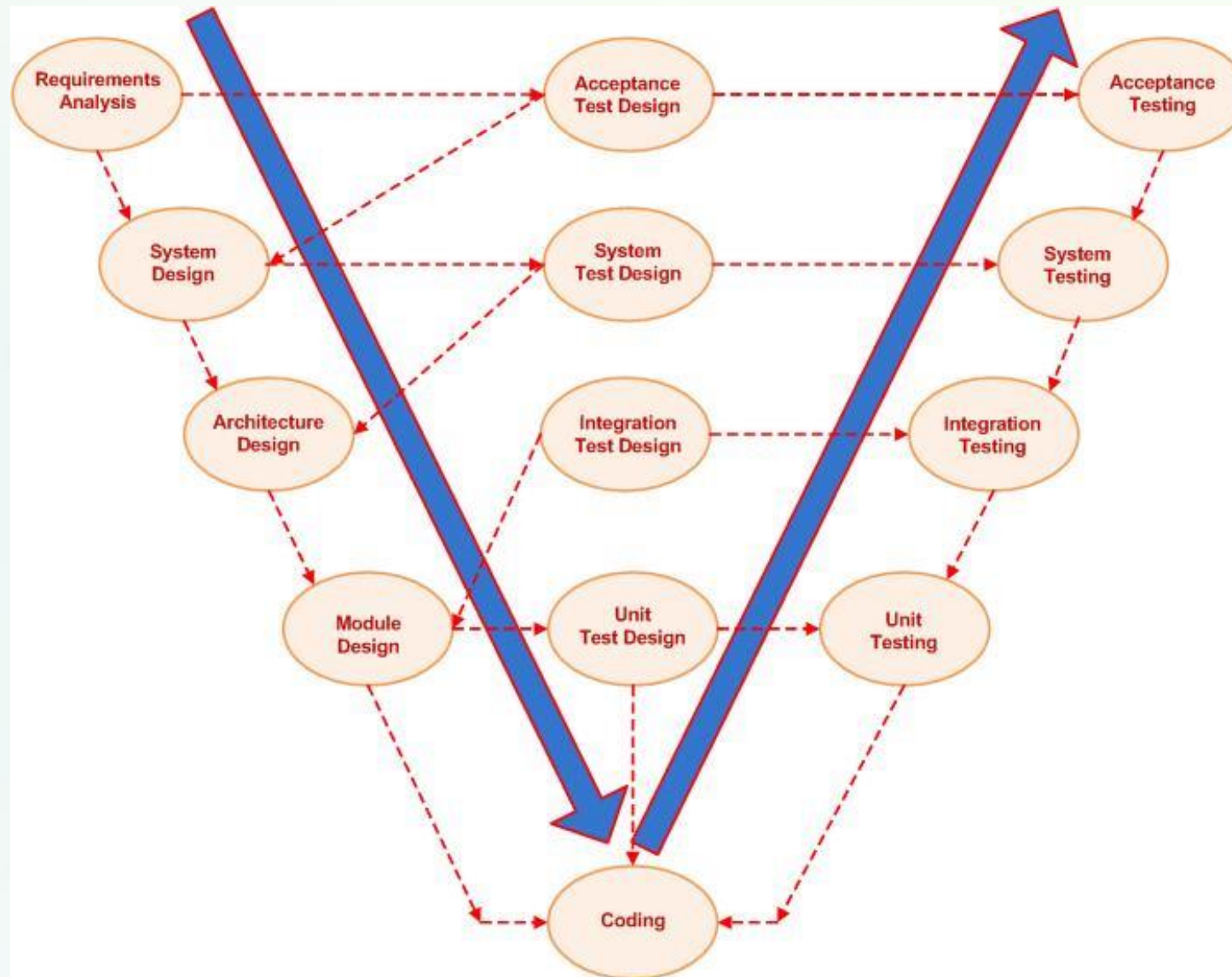
- **Black-Box** Testing without knowing internal code (e.g., Boundary Value Analysis, Equivalence Partitioning).
- **White-Box** Testing based on internal code structure (e.g., Statement Coverage).
- **Experience-Based** Using intuition and past knowledge (e.g., Error Guessing).

Test Case - Components

- ❖ A test case is a **detailed, step-by-step document** that specifies the **inputs, execution conditions, and expected outcomes** for a specific **test scenario**.
- ❖ It is the most granular level of testing documentation.

Test Case ID	• A unique identifier (e.g., TC-LOGIN-01).
Preconditions	• What must be true before the test starts (e.g., "User is logged out").
Test Steps	• Detailed actions to perform.
Test Data	• The specific inputs used (e.g., Username: testuser1).
Expected Result	• What the system should do if it's working correctly.
Actual Result & Status	• Filled out during execution (Pass/Fail).

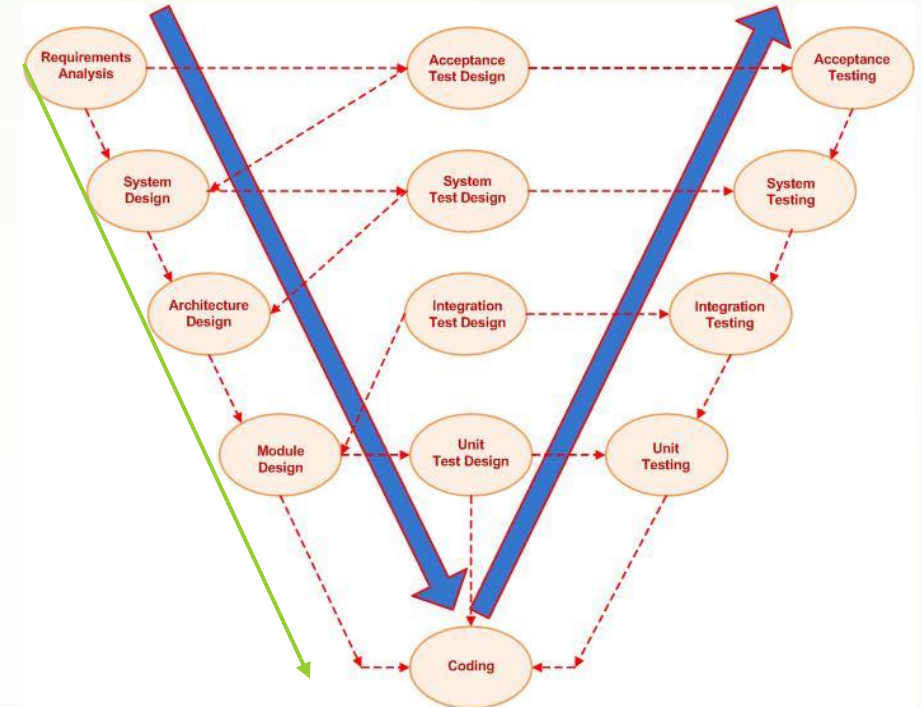
V & V Model



Verification Phase and Activities

❖ Software Reviews

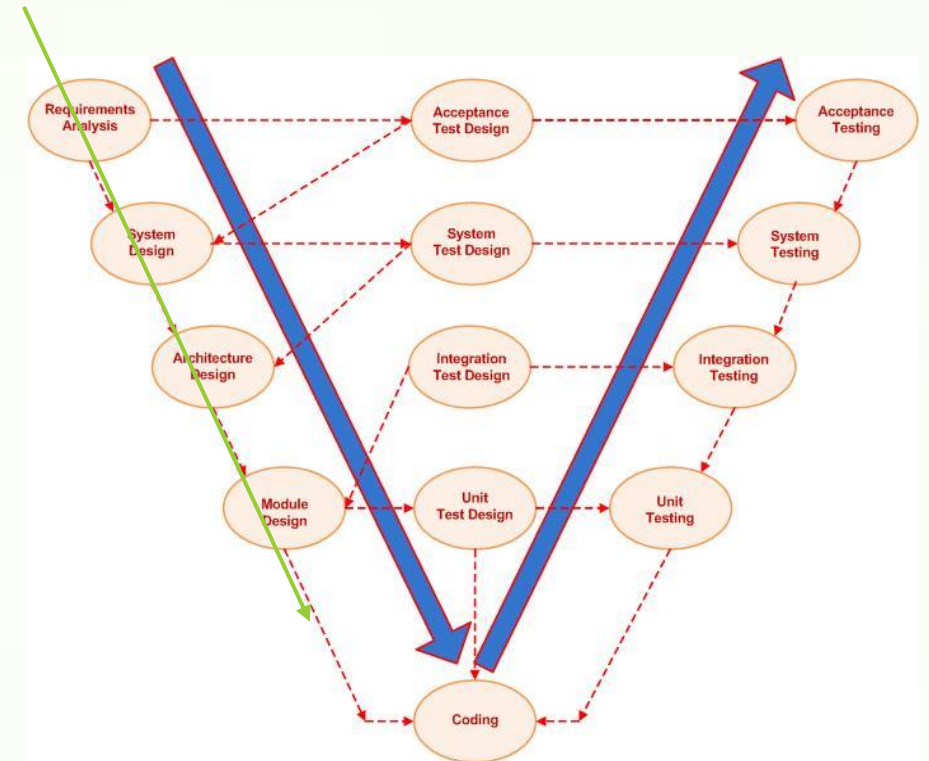
- ❖ A Review is the broadest category.
- ❖ It is a **systematic meeting** or process where a **software product (code, requirements, or design)** is examined by project personnel, managers, or users to find defects and provide feedback.
- ❖ Goal: To improve quality, validate requirements, and ensure the team is on the right track.



Verification Phase and Activities

❖ Software Reviews - Types

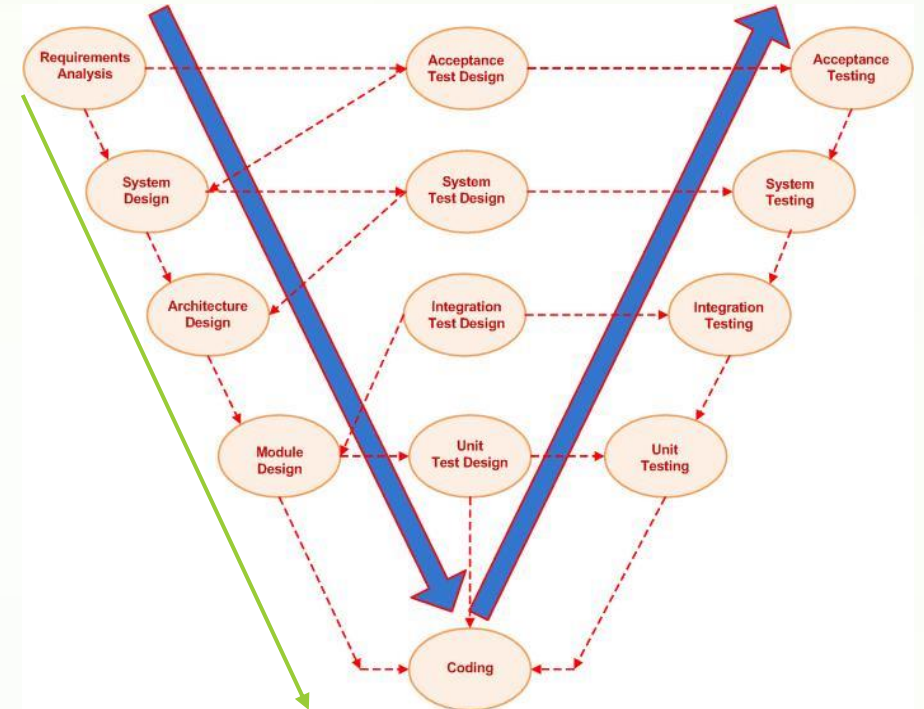
- ❖ **Peer Reviews:** Conducted by colleagues (e.g., Code Reviews, Walkthroughs).
- ❖ **Management Reviews:** Conducted by managers to monitor progress and make high-level decisions.
- ❖ **Formal vs. Informal:** Formal reviews follow a strict agenda and documented process, while informal reviews (like a "buddy check") are less structured.



Verification Phase and Activities

❖ Software Inspection

- ❖ An Inspection is the **most formal and rigorous type** of peer review.
- ❖ Originally defined by Michael Fagan at IBM, it follows a very specific, disciplined six-step process.
- ❖ The Process:



Planning



Overview



Preparation



Inspection Meeting



Rework

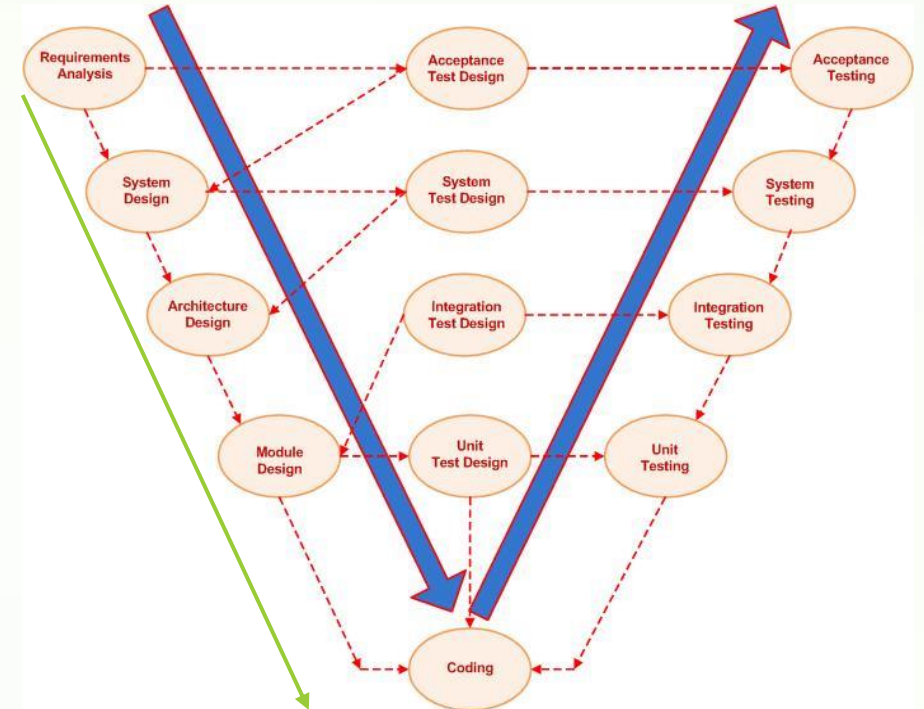


Follow-up

Verification Phase and Activities

❖ Software Inspection – Key Roles

- ❖ **Moderator:** Leads the meeting and ensures the process is followed.
- ❖ **Reader/Author:** Paraphrases the document for the group.
- ❖ **Recorder (Scribe):** Documents every defect found.
- ❖ **Differentiator:** It is data-driven, using checklists and entry/exit criteria to measure and improve the development process itself, not just the code.

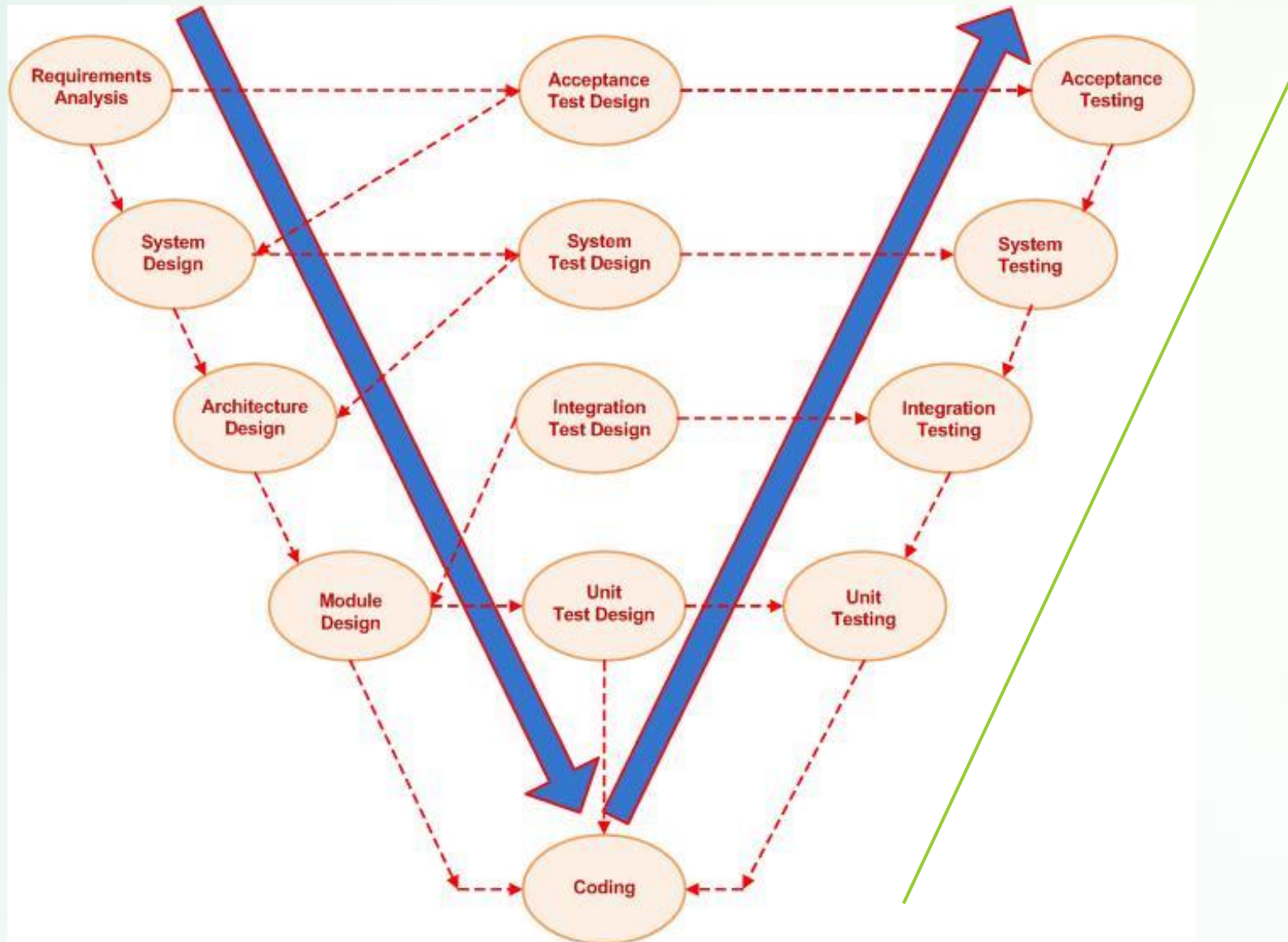


Verification Phase and Activities

❖ Software Auditing

- ❖ An Audit is an independent, objective examination conducted by people external to the development team (often an internal audit department or a third-party organization).
- ❖ **Focus** - Instead of looking for technical "bugs," auditors check for **compliance with standards** (like ISO/CMMI), **legal regulations**, or **internal policies**.
- ❖ **Scope** - Broad. It examines not just the code, but also management records, security protocols, and licensing agreements.
- ❖ **Outcome** - A formal Audit Report that outlines non-compliance issues and recommends corrective actions for the organization.

Validation Phase and Activities



V&V Model – Validation Phase Activities



V&V Model – Validation Phase Activities

Component (Unit) Testing

- ❖ Component testing (also called unit, module, or program testing) is the **testing of individual software components in isolation to detect defects and verify functioning of software modules, classes, or functions before integration**

V&V Model – Validation Phase Activities

Component (Unit) Testing

❖ Basis for Testing

- ❖ Program specifications (low-level design documents)
- ❖ Component design specifications
- ❖ Source code itself (for structural/white-box testing)
- ❖ Component requirements derived from system requirements
- ❖ Interface specifications for the component

V&V Model – Validation Phase Activities

Component (Unit) Testing - Process

Test Analysis

- Identify test conditions from component specifications and code structure

Test Design

- Create test cases using
 - **White-box techniques:** Statement coverage, decision coverage, condition coverage
 - **Black-box techniques:** Equivalence partitioning, boundary value analysis on component inputs

Test Implementation

- Developers write unit tests using frameworks (JUnit, NUnit, pytest)
- Create **stubs** (simulated called components) and **drivers** (simulated calling components) to isolate the unit

Test Execution

- Run tests automatically as part of build process (CI/CD pipeline)

Evaluation

- Measure code coverage metrics (statement, branch coverage)

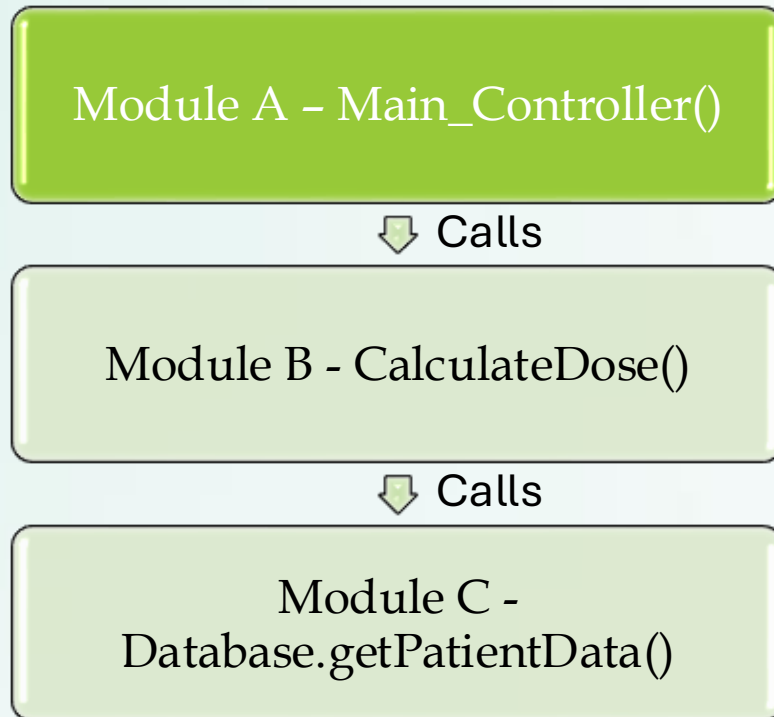
V&V Model – Validation Phase Activities

Component (Unit) Testing – Key Elements

Term	ISTQB Definition	Purpose	Used In Strategy
Stub	A test double that replaces a called component and returns predefined responses A Stub is a dummy module that replaces a lower-level module which is not yet developed.	Simulates lower-level components that are called by the component under test Used when testing a component that depends on others	Top-Down Integration
Driver	A test double that replaces a calling component and invokes the component under test with test data A Driver is a dummy module that calls the module under test	Simulates higher-level components that call the component under test Used when testing a component that has no caller yet	Bottom-Up Integration
Test Harness	Test environment comprising stubs, drivers, and other utilities to execute tests	Provides execution framework for isolated component testing	Both strategies

V&V Model – Validation Phase Activities

Component (Unit) Testing – STUB



```
1 # STUB replaces Database (dependency BELOW CalculateDose)
2 def getPatientData_stub():
3     return {"age": 65, "weight": 70} # Fixed test data
4
5 # Test CalculateDose WITH stub
6 def test_calculate_dose():
7     patient = getPatientData_stub() # ← STUB called HERE
8     dose = calculate_dose(patient["age"], patient["weight"])
9     assert dose == 35 # mg
```

Module C is not yet Developed

V&V Model – Validation Phase Activities

Component (Unit) Testing – Drivers

Module A is not yet Developed

Module A – Main_Controller()

↓ Calls

Module B - CalculateDose()

↓ Calls

Module C -
Database.getPatientData()

```
1  # DRIVER replaces MainController (caller ABOVE CalculateDose)
2  def driver_test_calculate_dose():
3      # DRIVER PROVIDES test inputs directly
4      dose = calculate_dose(age=65, weight=70)  # ← DRIVER calls HERE
5      print(f"Driver result: {dose} mg")
6      assert dose == 35
7
8  # Run the driver
9  driver_test_calculate_dose()
```

V&V Model – Validation Phase Activities

Aspect	Stub	Driver
Replaces	Called component (dependency)	Calling component (client)
Direction	Downward dependency	Upward dependency
Integration Strategy	Top-Down	Bottom-Up
Test Focus	"Does my component handle responses correctly?"	"Does my component produce correct outputs?"
Code Position	Injected INTO component under test	Wraps AROUND component under test
ISTQB Relationship	Used when testing from top down	Used when testing from bottom up

V&V Model – Validation Phase Activities

Integration Testing

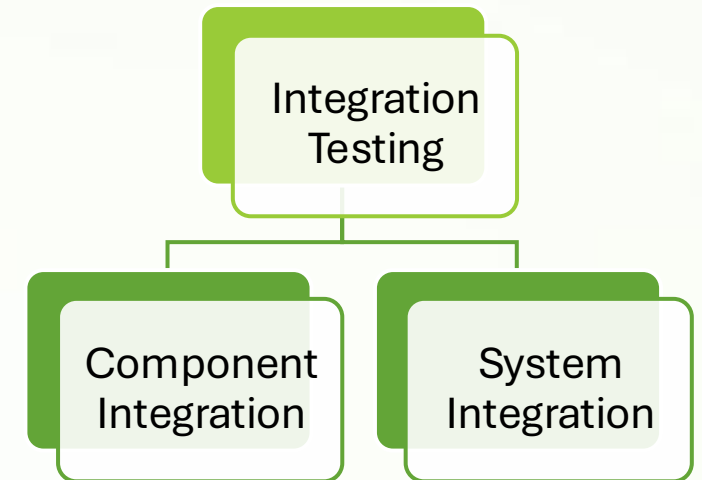
❖ Integration testing verifies interaction between modules. The goal is to **Detect interface defects, data flow issues, and interaction failures**

❖ Component Integration Testing

❖ Testing interactions between integrated components/modules

❖ System Integration Testing

❖ Testing interactions between integrated systems/subsystems



V&V Model – Validation Phase Activities

Integration Testing - Input

- ❖ Interface specifications (API contracts, message formats)
- ❖ Architectural design documents showing component relationships
- ❖ Database schema definitions
- ❖ Communication protocols (REST APIs, SOAP, messaging queues)
- ❖ System/subsystem design specifications

V&V Model – Validation Phase Activities

Integration Testing - Incremental Integration Testing

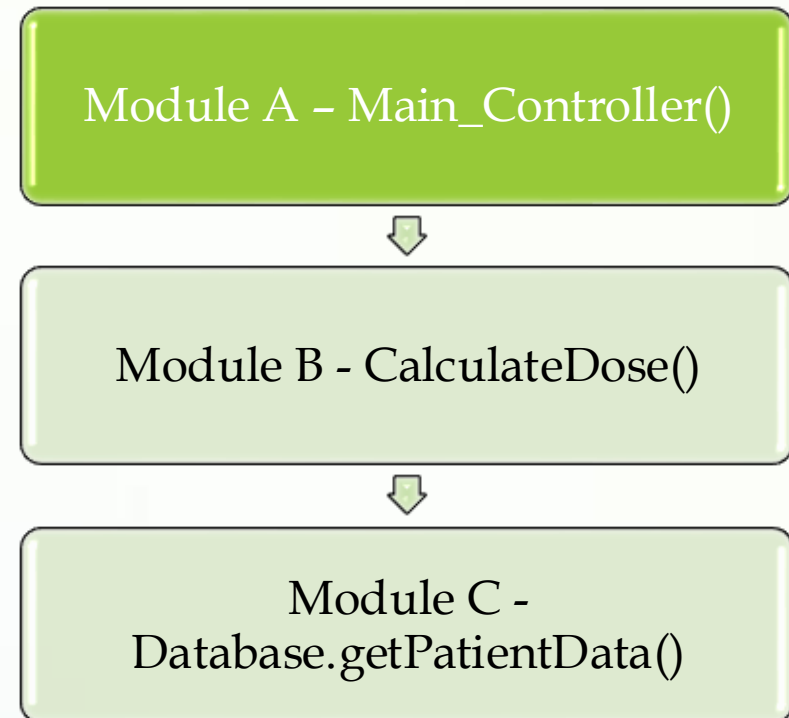
❖ Modules are **integrated step by step**. Three main approaches:

❖ Top-Down

❖ Bottom-Up

❖ Hybrid

❖ Consider the following Modules



V&V Model – Validation Phase Activities

Integration Testing – Top-Down Integration Testing

- ❖ Start testing from the top-level (main module) and integrate downward.
 - ❖ If lower modules are not ready → use stubs.

- ❖ **Create a Stub for Module C since it does not Exist**

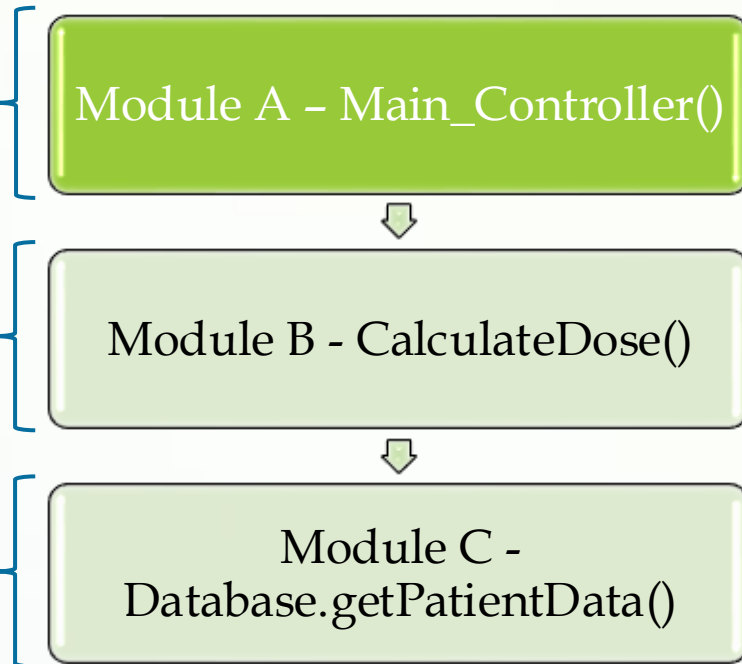
```
# Stub for Database  
def get_data():  
    return 10 # dummy value
```

- ❖ **Real Calculator**

```
def calculate(x):  
    data = get_data()  
    return x + data
```

- ❖ **Main Controller**

```
def main():  
    result = calculate(5)  
    print(result)  
main()
```



V&V Model – Validation Phase Activities

Integration Testing – Bottom-Up Integration Testing

- ❖ Start testing from lowest-level modules and integrate upward.
 - ❖ If upper modules are not ready → use drivers.

❖ Real Database Module

```
def get_data():  
    return 10
```

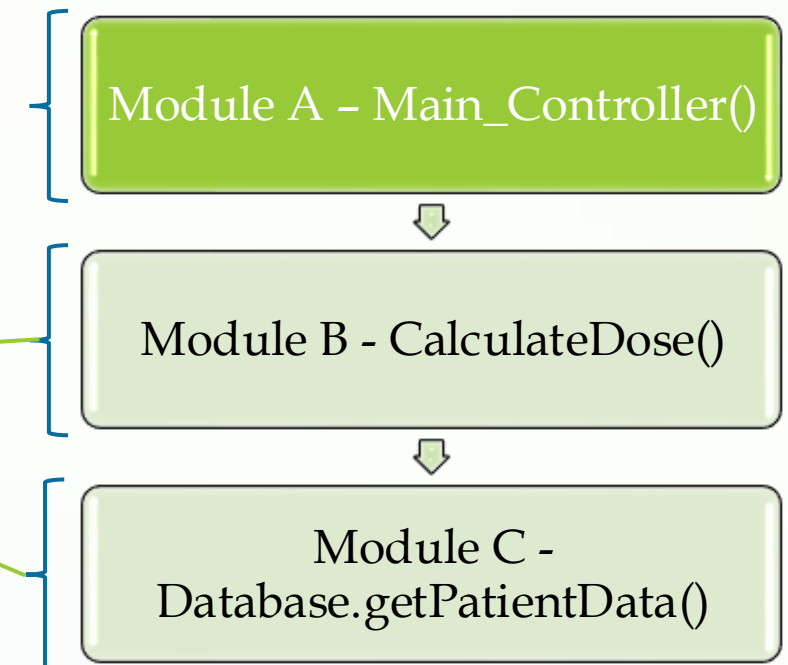
❖ Driver for Database

```
def driver():  
    value = get_data()  
    print("Database returned:", value)
```

```
driver()
```

❖ Integrate Calculator

```
def calculate(x):  
    return x + get_data()  
def driver():  
    print(calculate(5))  
driver()
```



V&V Model – Validation Phase Activities

Integration Testing – Interface Testing

❖ Check correctness of:

❖ Function parameters

❖ Data types

❖ Data formats

❖ Return values

Module A:

```
python

def get_age():
    return "25"    # string
```

Module B:

```
python

def calculate_birth_year(age):
    return 2025 - age
```

Integration:

```
python

age = get_age()
year = calculate_birth_year(age)
```

Error: cannot subtract string from integer

V&V Model – Validation Phase Activities

Integration Testing – Data Flow Testing

❖ Check whether correct data flows between modules

❖ Integration test reveals wrong data passed

```
def get_price():  
    return 100
```

```
def apply_discount(price):  
    return price * 0.9
```

```
def final_price():  
    price = get_price()  
    # Mistake: wrong variable used  
    return apply_discount(50)
```

V&V Model – Validation Phase Activities

Integration Testing

Strategy	How It Works	Stub/Driver Usage	Best For
Big Bang	All components integrated at once, then tested	None	Very small systems (<5 components)
Top-Down	Start with top-level components; stub lower modules	Stubs for missing lower components	Systems where major functionality is in upper layers
Bottom-Up	Start with lowest-level components; driver higher modules	Drivers for missing higher components	Systems requiring stable foundational services first
Sandwich/Hybrid	Combine top-down + bottom-up from middle layer	Both stubs & drivers	Medium/large systems

V&V Model – Validation Phase Activities

Integration Testing - System Integration Testing

- ❖ Testing interaction between different systems and sub systems completely including
 - ❖ External third-party systems (payment gateways, SMS providers)
 - ❖ Databases and middleware
 - ❖ Hardware devices (scanners, IoT sensors)
 - ❖ Legacy systems via adapters

V&V Model – Validation Phase Activities

Integration Testing - System Integration Testing

❖ When Performed

- ❖ After component integration testing completes
- ❖ Before system testing begins
- ❖ Typically owned by integration testers or QA team

❖ Inputs Required

- ❖ System interface specifications
- ❖ API documentation (OpenAPI/Swagger)
- ❖ Message formats (JSON/XML schemas)
- ❖ Network protocols (REST, SOAP, MQTT)
- ❖ Service Level Agreements (SLAs) for external systems

V&V Model – Validation Phase Activities

Integration Testing - System Integration Testing

Technique	Purpose	Context	Example
End-to-End (E2E) Flow Testing	Validate complete business process across systems	Primary SIT technique	Patient registration → Lab order → Results delivery across 3 systems
Contract Testing	Verify consumer-provider API compatibility	Critical for microservices	Provider publishes Pact contract; consumer tests against it before integration
Service Virtualization	Simulate unavailable external systems	Replaces stubs for complex dependencies	Virtualize payment gateway that charges \$0.01/test call
Message/Queue Testing	Validate asynchronous communication	For event-driven architectures	Verify order event published to Kafka is correctly consumed by inventory system
Database Integration Testing	Test data consistency across systems	Often overlooked	Verify order created in System A appears correctly in System B's reporting DB

V&V Model – Validation Phase Activities

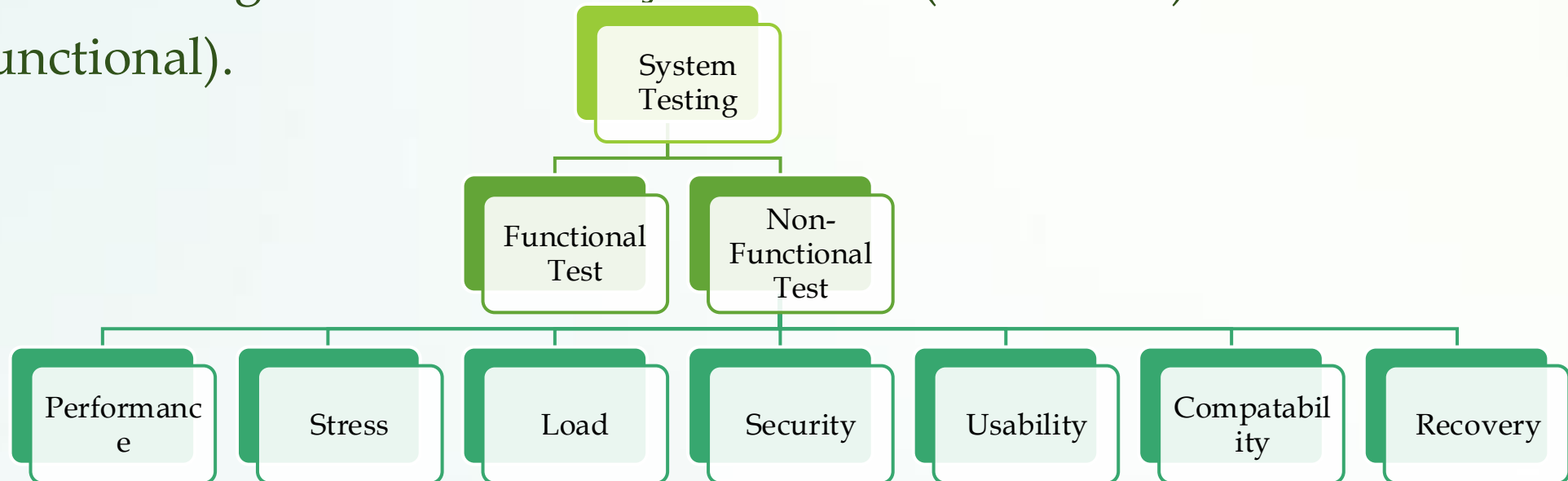
Integration Testing - Component Vs System Integration Testing

Aspect	Component Integration Testing	System Integration Testing
Scope	Components within one system	Multiple independent systems
Boundaries	Internal interfaces (method calls, internal APIs)	External interfaces (REST APIs, message queues, files)
Environment	Developer workstation / CI pipeline	Integration test environment (mirrors production topology)
Dependencies	Other components (stubs/drivers used)	External systems (service virtualization used)
Defect Focus	Data corruption at interface boundaries	System availability, protocol compliance, data consistency
ISTQB Level	Component Integration Testing	System Integration Testing
Typical Tools	JUnit + Mockito (stubs/mocks)	Postman, Pact, WireMock, Kafka test containers

V&V Model – Validation Phase Activities

System Testing

- ❖ It is testing an integrated system to **evaluate compliance with specified requirements**.
- ❖ It **validates the complete, integrated system** in an environment that mirrors production – testing both **what the system does** (functional) and **how well it does it** (non-functional).



V&V Model – Validation Phase Activities

System Testing - Input

- ❖ Before system testing begins, these artifacts must exist:
 - ❖ System Requirements Specification (SRS)
 - ❖ Functional requirements (use cases/user stories)
 - ❖ Non-functional requirements (performance, security specs)
 - ❖ Interface specifications (API contracts)
 - ❖ Business rules documentation

V&V Model – Validation Phase Activities

System Testing – Functional Testing

Technique	When to Use	Tiny Example
Equivalence Partitioning	Input fields with valid/invalid ranges	Password field (8-64 chars): • Valid: "Pass1234" (12 chars) • Invalid: "short" (5 chars) → reject
Boundary Value Analysis	Numeric/text limits	Age field (18-120): • Test: 17 (reject), 18 (accept), 19 (accept), 119 (accept), 120 (accept), 121 (reject)
Decision Table	Complex business rules	Discount rules: • Premium + >\$100 → 15% off • Standard + >\$100 → 10% off • Test all 4 rule combinations
State Transition	Workflow/status changes	Order status flow: CREATED → PAID → SHIPPED → DELIVERED Test: Can't go PAID → DELIVERED (skip SHIPPED)